



인자의 전달과 메모리 구조(1)

- main() 함수가 시작되고 화살표까지 실행된 순간의 메모리 상태

```
int max(int a, int b);

int main()
{
    // 3과 5중에 큰 값을 구한다.
    int arg1 = 3;
    int arg2 = 5;
    int ret = max(arg1, arg2);

    return 0;
}

int max(int a, int b)
{
    return a > b ? a : b;
}
```

[그림 14-15]

인자의 전달과 메모리 구조(2)

- max() 함수가 호출되고 화살표까지 실행된 순간의 메모리 상태



```
int max(int a, int b)
{
    return a > b ? a : b;
}

int main()
{
    // 3과 5중에 큰 값을 구한다.
    int arg1 = 3;
    int arg2 = 5;
    int ret = max(arg1, arg2);

    return 0;
}
```

[그림 14-16]

- 이때 다음과 같은 가상의 코드가 실행된다.

```
int a = arg1;
int b = arg2;
```

인자의 전달과 메모리 구조(3)

- max() 함수가 끝나고 화살표까지 실행된 순간의 메모리 상태

```
int max(int a, int b)
{
    return a > b ? a : b;
}

int main()
{
    // 3과 5중에 큰 값을 구한다.
    int arg1 = 3;
    int arg2 = 5;
    int ret = max(arg1, arg2);
    return 0;
}
```

[그림 14-17]

포인터 변수를 인자로 전달하기(1)

- 포인터 변수를 사용해서 함수 밖으로 결과를 전달하는 예

```
void GCD_LCM(int a, int b, int* pgcd, int* plcm)
{
    // 유클리드 호제법을 사용해서 GCD를 구한다.
    // 중간 소스 코드 생략 (p.321 참조)

    // 결과를 저장한다.
    *pgcd = y;
    *plcm = a * b / *pgcd;
}

int main()
{
    // 28과 35의 최대공약수와 최소공배수를 구한다.
    int gcd = 0;
    int lcm = 0;
    GCD_LCM( 28, 35, &gcd, &lcm);

    return 0;
}
```

- GCD_LCM() 함수가 호출될 때 실행되는 가상의 코드

```
int a = 28;
int b = 35;
int* pgcd = &gcd;
int* plcm = &lcm;
```

포인터 변수를 인자로 전달하기(2)

- 화살표까지 실행된 순간의 메모리 상태

```
void GCD_LCM(int a, int b, int* pgcd, int* plcm)
{
    // 유클리드 호제법을 사용해서 GCD를 구한다.
    // 중간 소스 코드 생략
    // 결과를 저장한다.
    *pgcd = y;
    *plcm = a * b / *pgcd;
}

int main()
{
    // 28과 35의 최대공약수와 최소공배수를 구한다.
    int gcd = 0;
    int lcm = 0;
    GCD_LCM( 28, 35, &gcd, &lcm);

    return 0;
}
```

[그림 14-21]

포인터 변수를 인자로 전달하기(3)

- 화살표까지 실행된 순간의 메모리 상태

```
void GCD_LCM(int a, int b, int* pgcd, int* plcm)
{
    // 유클리드 호제법을 사용해서 GCD를 구한다.
    // 중간 소스 코드 생략 (p.321 참조)

    // 결과를 저장한다.
    *pgcd = y;
    *plcm = a * b / *pgcd;
}

int main()
{
    // 28과 35의 최대공약수와 최소공배수를 구한다.
    int gcd = 0;
    int lcm = 0;
    GCD_LCM( 28, 35, &gcd, &lcm);

    return 0;
}
```

[그림 14-22]



포인터 변수를 인자로 전달하기(4)

- 포인터 타입의 인자를 사용해서 함수의 결과 값을 얻어오는 방법의 정리
 - 함수의 매개 변수는 포인터 타입으로 정의한다.
 - 인자를 넘겨줄 때는 결과 값을 담고 싶은 변수의 주소를 넘겨준다.
 - 함수 안에서 결과를 넘겨줄 때는 매개변수가 가리키는 곳에 값을 넣어준다.

레퍼런스 변수를 인자로 전달하기(1)

- 레퍼런스 변수를 사용해서 함수 밖으로 결과를 전달하는 예

```
void GCD_LCM(int a, int b, int& gcd, int& lcm)
{
    // 유클리드 호제법을 사용해서 GCD를 구한다.
    // 중간 소스 코드 생략 (p.325 참조)

    // 결과를 저장한다.
    gcd = y;
    lcm = a * b / *pgcd;
}

int main()
{
    // 28과 35의 최대공약수와 최소공배수를 구한다.
    int gcd = 0;
    int lcm = 0;
    GCD_LCM( 28, 35, gcd, lcm);

    return 0;
}
```

- GCD_LCM() 함수가 호출될 때 실행되는 가상의 코드

```
int a = 28;
int b = 35;
int& gcd = gcd;    // 뒤쪽의 gcd는 main() 함수 안에 있는 gcd를 말한다
int& lcm = lcm;    // 뒤쪽의 lcm는 main() 함수 안에 있는 lcm를 말한다
```




레퍼런스 변수를 인자로 전달하기(3)

- 레퍼런스 타입의 인자를 사용해서 함수의 결과 값을 얻어오는 방법의 정리
 - 함수의 매개 변수는 레퍼런스 타입으로 정의한다.
 - 인자를 넘겨줄 때는 결과 값을 담고 싶은 변수를 그대로 넘겨준다.
 - 함수 안에서 결과를 넘겨줄 때는 매개 변수에 값을 넣어준다.

배열을 인자로 전달하기(1)

- 배열 타입의 인자는 실제로는 포인터를 사용해서 전달된다.

```
int main()
{
    char array[20] = "Hello, World!";
    UsingArray( array);
    cout << "In main() : " << array << "\n";
    return 0;
}

void UsingArray(char arr[] ) // char* arr
{
    cout << "In UsingArray() : " << arr <<
        "\n";
    arr[12] = '?'; // 문자 '?'
}
```

[그림 14-27]

- UsingArray() 함수가 호출될 때 실행되는 가상의 코드

```
char* arr = array;
```

- 실행 결과

[그림 14-26]

스택에 만든 메모리는 반환할수 없다 !!!
can't return value in stack memory



배열을 인자로 전달하기(2)

스택에 만든 메모리는 반환할수 없다 !!!

can't return value in stack memory
dynamic allocation must be return!

- 배열을 인자로 전달하는 방법의 정리
 - 매개 변수의 타입을 적어줄 때 ‘배열의 원소 개수’는 적지 않는다.
 - 인자로 넘겨줄 때는 배열의 이름을 넘겨준다.
 - 인자로 넘어온 배열을 사용할 때는 그냥 평범한 배열을 사용하듯이 하면 된다.
- 요소의 개수를 전달할 것!, 특히 int 배열인 경우.